

SPRÁVA PAMĚTI - Kompletní Výpisy z Přednášky

1. ZÁKLADNÍ LAYOUT 32bitového SYSTÉMU

32bitová paměť = 4GB

Dělí se na:

- **User Space (0-3GB)** - IZOLOVANÝ PER PROCES (každý proces má svůj)
- **Kernel Space (3-4GB)** - SDÍLENÝ VŠEM PROCESŮM

Důsledek: Dva procesy si nemohou vzájemně sahat do paměti, ale oba sdílí stejný kernel space.

2. PAGE TABLE ENTRY - Flags (20 bitů)

Povinný bit:

- **Validity Bit (V bit):**
 - = 1 → stránka je PLATNÁ → překládá se
 - = 0 → stránka není v paměti → OS musí řešit:
 - Přidělit stránku a načíst ji
 - Nebo program se chybou ukončí

Volitelné bity:

- **Read bit (R)** - čtení povoleno
- **Write bit (W)** - zápis povoleno
- **Execute bit (X)** - spuštění povoleno (analogie: chmod u Linuxu)
- **Access bit (A)** - byla stránka čtena/psána (sleduje aktivitu)
- **Dirty bit (D)** - byla stránka MODIFIKOVÁNA (změnil se obsah)
- **Privilege bit** - uživatelské vs. kernelové práva

3. PANIENSKÝ RÁMEC - Co se stane?

Panienský rámec = paměť která byla právě alokována (malloc) ale ještě nemá obsah

Řešení OS:

1. Nejde se do ní zápsat přímo
2. Nejdřív se **nuluje** (zeroing) - vyplní se nulami
3. Validity bit = 0 (nejsou používaná data)
4. Access bit = 0 (nebyla přístupná)
5. **Výsledek: MINOR FAULT** - žádná disk I/O, jen inicializace paměti

4. PROCES KRÁDĚ STRÁNKY

Krok 1: Zjistíme že RAM je plná

→ Musíme někomu vzít stránku (oběť)

Krok 2: Najdeme oběť - kontrola ACCESS BIT

Access Bit ukazuje:

- Access Bit = 1 → byla čtena nedávno → ŠPATNÁ kandidátka (používá se)
- Access Bit = 0 → nebyla čtena → DOBRÁ kandidátka (nepoužívá se)

Krok 3: Kontrola DIRTY BIT - rozhodnutí co udělat

Čistá stránka (Dirty Bit = 0):

- Je to zrcadlo souboru na disku
- Uložíme ji tam kde jsme ji vzali (do souboru)
- Nebo: není swap potřeba
- = **MINOR FAULT** (žádná disk I/O)

Špinavá stránka (Dirty Bit = 1):

- Byla modifikována v RAM
- **MUSÍME JI ULOŽIT!** Dvě možnosti:

A) Ze souboru (memory-mapped file):

- Uložíme ji zpět do souboru odkud jsme ji vzali

B) Z mallocu (anonymní paměť):

- Uložíme do swap prostoru (OS si ho v paměti už rezervoval při alokaci)

- Uložíme do **swap prostoru** (OS si ho v paměti už rezervoval při alokaci)
- Když proces později přistoupí → vrátíme ze swapů
→ = **MAJOR FAULT** (DISK I/O! Pomalé!)

5. KLÍČ: KRÁDEŽ SPOČÍVÁ V ULOŽENÍ PŮVODNÍHO STAVU!

PRVNÍ KROK (nejdůležitější):

1. Zjistíme Access bit → víme zda se používá
2. Zjistíme Dirty bit → víme zda je změněná
3. Rozhodujeme se kde to uložit:
 - Čistá z disku? → Jen vymaž
 - Špinavá z souboru? → Ulož do souboru
 - Špinavá z malloc? → Ulož do swapů

DRUHÝ KROK:

- Označíme stránku jako "pryč" v page table
- Uložíme info kam se přesunula (swap offset, file offset)

6. TYPY PAGE FAULTŮ

MINOR FAULT (bez disk I/O)

- Panienská stránka (nulování)
- Čistá stránka z disku (jen ji vymaž)
- Access Bit = 0 (nebyla čtena)
- **Rychlé!**

MAJOR FAULT (s disk I/O)

- Špinavá stránka (musím ji uložit na disk/swap)
- Access Bit = 1 (byla čtena, ale teď ji kradu)
- Disk I/O = 5ms (vs 5ns v RAM) = 1 000 000× pomalejší
- **KATASTROFA PRO VÝKON!**

7. STRATEGIE VÝBĚRU OBĚTI - 3 ALGORITMY

1. NÁHODNÁ VOLBA

- Vyber náhodně
- Díky statistice průměr
- **Nevýhoda:** 50% šance na PAGE FAULT okamžitě
- **Nepoužívá se v praxi**

2. FIFO (First In First Out) - PRINCIP ČASOVÉ ŠIPKY

- Nejstarší stránka se vybere (která se nejdéle nepoužívá)
- **Nevýhoda:** neví nic o AKTUÁLNÍ aktivitě
- Nerespektuje že proces stránku právě používá

Praktické příklady:

- Obchod s oblečením: "Nový sortiment, staré věci z regálu ven"
- Knihovna: "Odstranění knihy která je dlouho neprodejná"
- Starší Windows (Win 8 a méně)

Optimalizace pro SSD disky:

- Místo bajt po bajtu, zapisuj celé megabajty
- Stránky vedle sebe se zapisují dohromady → efektivnější

Problém v zaplněném systému:

- Procesy si navzájem krádou stránky
- Nejhorší je vyřešit ukládání

Working Set Manager:

- Každý proces má přidělenou pracovní množinu stránek
- Problém: Když se proces přepne do jiného režimu (CPU mode), neví manager že teď potřebuje víc paměti
- Manager reaguje se zpožděním (exposed)

FIFO jako "socialistické hospodářství" - centrální přidělování, ale nefunguje stejně :D

3. LRU (Least Recently Used) - OPTIMÁLNÍ

- Stránka která byla **nejdéle nepoužita** (nečtena/nepsána)
- Respektuje **AKTUÁLNÍ aktivitu**
- Minimální PAGE FAULTS
- Používá se v praxi Linux, Windows (Windows 10 a výše)

- Používá se v praxi: LINUX, WINDOWS (WINDOWS 10 a výše)

Implementace:

Ideálně:

- Časové razítko při každém přístupu
- Při kráždí: najdi nejstarší timestamp

Prakticky (přes Access Bit):

- Každých N milisekund se vynulují Access bity
- Stránky bez Access bitu se krádo (nebyly v poslední chvíli přístupné)
- Efektivní, nenímu disk I/O

Jak to funguje:

- Access Bit = 1 → "Byl jsem čten nedávno"
- Access Bit = 0 → "Netouchali mě dlouho"
- Krádež: vezmi stránku s Access Bit = 0

Knihovna jako analogie:

- Vezmu knihu z knihovny, dám ji na začátek přihrádky
- Když chci odstranit, vezmu tu která je na konci (nejdéle se neodkládala)

LRU jako "kapitalismus" - "Začneš krást a přizpůsobíme se" :D

8. WORKING SET + PRINCIP LOKALITY

Working Set = sada stránek kterou proces aktivně používá v daný moment

Princip Lokality:

- Když proces čte stránku → brzy čte STEJNOU stránku nebo VEDLE
- Stránky jsou blízko sebe v adresovém prostoru
- Nejsou nikde extra jinde

Příklad:

```
for (int i = 0; i < 1000; i++) {
    x = array[i];    // Stránka A
    y = array[i+1];  // Stránka A (vedle!)
    z = array[i-1];  // Stránka A (vedle!)
}
```

LRU to rozumí → drží stránku v RAM, nevrací ji do swapů, protože ji bude brzy potřebovat znovu.

9. OPTIMIZACE STRÁNKOVÁNÍ

Slovní (page table) cachování:

- Způsobí dalekší swapování než bychom měli
- Když hodně swapujeme, nápojují se slovníky dřív

TLB (Translation Lookaside Buffer):

- Malá cache pro nejčastěji používané page table entries
- Když přestoupí na jinou část paměti, TLB miss = Horror

Zásobník (stack):

- K chování lokality friendly
- Snižuje počet page faultů

10. PŘÍKLADY NA ZKOUŠKU

Učitel bude chtít konkrétní příklady:

- **Obchod:** sortiment, staré věci z regálu ven (FIFO)
- **Knihovna:** senzáční knihy dostupné, nudné na konci (LRU)
- **Tabulkový kalkulačtor:** stejný kód všech procesů → sdílení DLL
- **Automobil:** dva lidi, jeden klíč → copy-on-write

11. SDÍLENÍ PAMĚTI

Jaké jsou 3 strategie sdílení?

Pozn.: Učitel neřekl název, pojd' vysvětlit princip

Co se vlastně sdílí?

Dva procesy si plánují regiony v adresovém prostoru, které chtějí sdílet.

Klíčové: Zatím se sdílí nic! (nejsou validní)

OS si jen "pamatuje" že to musí zajistit.

Sdílení se projeví jen když:

- 2 STRÁNKY používají STEJNÝ RÁMEC ve STEJNÝ ČAS

- Jinak je to jen potenciální sdílení

Režim 1: READ ONLY

Případ: Kódový region

- Všechny instance programu (tabulkový kalkulátor) používají stejný kód
- Kód se nemění → read only → můžeme sdílet

Případ: DLL knihovny

- Všichni si DL knihovnu namapují jako sdílenou (shared library)
- Při spuštění: "LOAD DLL"
- Všichni používají stejný kód v paměti

Problém s DLL:

- Voláním funkce přes DLL je pomalejší (dynamické) než statické volání
- Až 2× pomalejší
- Dnes je to optimalizované, všichni to používají DLL
- **Chybou je** používat elementární funkce přes DLL (např. scan_pixel)

12. COPY-ON-WRITE PRISTUP

Idea: Lazy approach (lenivé čekání)

Máme proces A s nějakým blokem paměti. Proces B si ho chce zkopírovat.

Naivní přístup:

- Bajt po bajtu kopírujeme
- Nebo po více bajtech, ale postupně
- Problém: Všechny verze stránky se postupně dostávají do pracovní množiny
- Každých 4KB → spadne 2 stránky (!)
- **Snižuje to efektivitu kopírování**

Lepší: Copy-on-Write

1. Máme blok paměti
2. Nasdílíme ho na druhé místo (bez fyzické kopie!)
3. Oba procesy vidí STEJNOU paměť
4. Když se oba čtou → všechno je OK
5. Problém: Když jeden zapíše a druhý by měl vidět starý obsah?

Řešení:

- Když se jeden pokusí ZAPSAT do druhého → OS oddělí stránky
- Vytvoří se nová stránka bez té blbě změny
- Fyzická oddelenost až je potřeba (lazy!)

Lenivost dovedena k dokonalosti:

- Čekáme dokud se projekt NEZAPÍŠE
- Neodděláme
- Až jeden přistoupí k ZÁPISU → zajistíme fyzickou oddelenost

Čistý model - VŠECHNY STRÁNKY NEPLATNÉ

Pracujeme s nevalidními stránkami:

1. Proces A přistoupí na stránku → vypadek → se mapuje na rámec X
2. Proces B taky přistoupí na stejnou stránku → vypadek → mapuje se na stejný rámec X
3. **Nyní sdílíme!**
4. Když B zapíše do rámce X:
 - Vytvoří se nový rámec Y
 - B se nyní mapuje na Y
 - Obsah Y je nový
 - Obsah X zůstane nezměněn (vrátí se ze swapů?)
5. Když A zapíše a B se mapuje na Y?
 - Obnovuje se původní obsah přes swap?
 - **Lenivost v lenivosti!**

13. ANALOGIE S AUTOMOBILEM

Scénář: Dám 2 lidem klíče od auta

FIFO přístup:

- Jakmile chcou jet na jiné místo → musím auto zduplikovat
- Každý má své auto

Copy-on-Write přístup:

- Dám jim klíče

- **Auto neexistuje dokud ho nechceji použít!**
- Auto vyrobíme až ho někdo chce použít
- Když to není konflikt → stačí jedno
- Když se jejich cesty liší → vyrobíme druhé

Rezultát: Perfektně správně!

14. VYTVÁŘENÍ PROCESŮ V UNIXU vs WINDOWS

UNIX:

- `fork()` vytváří nový proces (dětský proces)
- Dítě je kopií rodiče (včetně paměti)
- Je to RYCHLÉ díky copy-on-write

WINDOWS:

- Používá se spíš VLÁKNA (threads)
- Vlákna sdílí paměť přímo (bez copy-on-write)
- Menší režie, ale těžší na synchronizaci

KLÍČOVÉ POZNATKY NA ZKOUŠKU

1. **32bit = 4GB: 3GB user + 1GB kernel**
2. **Validity bit je povinný**, ostatní jsou pro optimalizaci
3. **FIFO = princip časové šipky**, LRU = optimální v praxi
4. **Access Bit sleduje aktivitu** → LRU v praxi
5. **Dirty Bit řeší kam se stránka uloží**
6. **Copy-on-Write = lazy approach** (neodděláme dokud není potřeba)
7. **Working Set = sada aktivních stránek procesu**
8. **Princip Lokality = base pro LRU úspěch**
9. **DLL = sdílení kódu, read-only**
10. **Procesy si mohou krást stránky v zaplněném systému**

Mocniny 2 v jednotkách datového úložiště

Bínární (IEC - informatické jednotky)

Jednotka	Symbol	Mocnina 2	Bajty	Hodnota
Bit	bit	2^0	0,125	1 bit
Bajt	B	2^3	1	8 bitů
Kilobajt	KB	2^{10}	1 024	1 024 B
Megabajt	MB	2^{20}	1 048 576	1 024 KB
Gigabajt	GB	2^{30}	1 073 741 824	1 024 MB
Terabajt	TB	2^{40}	1 099 511 627 776	1 024 GB
Petabajt	PB	2^{50}	1 125 899 906 842 624	1 024 TB
Exabajt	EB	2^{60}	1 152 921 504 606 846 976	1 024 PB
Zettabajt	ZB	2^{70}	1 180 591 620 717 411 303 424	1 024 EB
Yottabajt	YB	2^{80}	1 208 925 819 614 629 174 706 176	1 024 ZB

1.

32 bitová paměť -> 4giga